Command Guide

How to run Eagle's Eye safely from VS Code · October 2026

Short answer. No. These commands do not touch your website's code, files or server. **npm install** only adds packages inside the eagle-eye folder. **npm run eagle -- crawl** only reads a website over the web, like a visitor would, and saves what it read to one file inside eagle-eye. The one way to cause a problem is running them from the wrong folder, so section 1 comes first.

1. Before you run anything: be in the right folder

Everything in this guide is run from the **eagle-eye** folder, never from your website's folder. In VS Code, use **File, Open Folder** and choose eagle-eye, or open a terminal and change into it. Then check where you are:

```
cd C:¥Users¥godca¥Downloads¥projects-longterm¥eagle-eye
dir package.json
```

You should see a package.json whose name is eagles-eye. If the terminal prompt shows any other project folder (for example your AIIT website), stop and change folder first. **Requirement:** Node.js 24 or newer. Check with node -v.

2. Commands at a glance

Command	What it does	What it touches
npm install	Downloads the libraries Eagle's Eye needs.	Creates eagle-eye/node_modules (about 100 MB). Nothing outside the folder.
npm test	Runs the 381 automated tests.	Starts a tiny local test server on your computer for a few seconds. No internet, no files kept.
npm run typecheck	Checks the code for type errors.	Reads files only.
npm run eval	Scores answer quality on the built-in sample university.	Local test server; writes a report to evals/reports.
npm run eagle -- crawl <url>	Reads the pages of a website and indexes them.	Sends read-only requests to that website; writes eagle-eye/data/eagle-cli.sqlite.
npm run eagle -- crawl <url> --render	Same, but runs the page's JavaScript first (React/Vue sites that arrive empty).	Starts a hidden Chromium browser on your computer. Needs a one-time download.
npm run demo -- <url>	Reads a site, then serves a page with the real chat widget answering from it.	Sends read-only requests to that site; writes eagle-eye/data/demo.sqlite; serves http://localhost:8788 on your computer only.
npm run build	Builds the SDK, the chat widget and the React component.	Writes dist folders inside the packages.
npm run eagle -- ask "..."	Answers one question from the indexed site.	Reads the local database. No internet.
npm run eagle -- chat	Interactive chat that remembers follow-ups.	Reads the local database. No internet (unless you set a language model).
npm run eagle -- search "..."	Shows the raw passages found for a query.	Reads the local database.
npm run eagle -- pages	Lists every page that was indexed.	Reads the local database.

3. Step by step

Step 1. Install (once)

```
npm install
```

You may see a line about SQLite being experimental. That is a harmless Node notice. Re-running npm install later is safe.

Step 2. Confirm everything works (optional but recommended)

```
npm test
npm run eval
```

Expect 381 passed and All Phase 1 targets met. If either fails on your machine, send me the output.

Step 3. Index a website

Start small so you can see results quickly and keep the load on the target site tiny:

```
npm run eagle -- crawl https://your-education-site.edu --max-pages 30
```

It prints each page as it is indexed, then a summary. When it looks right, raise the limit (for example --max-pages 200) and run it again. The second run is incremental and only re-reads pages that changed.

Step 3b. If the crawl indexes 0 pages (React / Vue / single-page sites)

Some sites send an empty page and draw everything with JavaScript, so there is nothing to read. Install the browser once, then add --render:

```
npx playwright install chromium
npm run eagle -- crawl https://your-react-site.org --max-pages 30 --render
```

Expect about 3 to 6 seconds per page, so keep --max-pages small at first. The browser blocks any attempt by the page to reach internal or private addresses.

Step 4. Ask questions

```
npm run eagle -- ask "How much is tuition for computer science?"
npm run eagle -- ask "What is the application deadline?" --debug
npm run eagle -- chat
```

--debug shows how the answer was found. In chat, type exit to leave. Answers are quoted from the site with a source link, or the assistant says it could not find the answer. **Write down every wrong answer and every wrongly declined question.** That list is the most valuable input for improving the engine.

Step 5. Look under the hood

```
npm run eagle -- search "scholarship deadline"
npm run eagle -- pages
```

Step 6. Score it on your own questions (optional)

Copy evals/datasets/education-demo.json to a new file, replace the questions and expected pages with ones for your site, then run:

```
npm run eval -- --url https://your-education-site.edu --dataset my-questions.json
```

Step 7. Use a real language model with Ollama (recommended)

Ollama is **free** and runs entirely on your computer. You do **not** need to sign up or log in. Install it once (the Windows installer from ollama.com, or winget install -e --id Ollama.Ollama --source winget), then download the model. This model is Apache-2.0 licensed, so it is fine for commercial use, and it is about 2.5 GB:

```
ollama pull qwen3:4b-instruct
```

Then tell Eagle's Eye to use it. In PowerShell (these settings last for that terminal window):

```
$env:EAGLE_LLM_BASE_URL = "http://localhost:11434/v1"
$env:EAGLE_LLM_MODEL = "qwen3:4b-instruct"
npm run eagle -- llm-check
npm run eagle -- chat
```

llm-check should print OK and say the model answers directly. The first answer after a pause takes about 10 seconds while the model loads; later answers take roughly 3 to 10 seconds on a laptop with a small GPU. **Use a non-thinking model.** Models that show their reasoning (the plain qwen3:4b, DeepSeek-R1) are slower and can leak that reasoning into answers.

Step 8. See the chat widget on a website

This reads a site and opens a page with the real chat window answering from it, exactly as a customer's visitors would see it. Add --render for React or Vue sites, and set the model variables from step 7 for conversational, streamed answers:

```
npm run demo -- https://www.reed.edu --max-pages 40
npm run demo -- https://aiit.network --render --max-pages 20
```

When it prints Open http://127.0.0.1:8788, open that address in your browser. Press Ctrl+C in the terminal to stop it. Everything stays on your computer. Try adding ?theme=dark, ?position=left or ?color=%231b6b4a to the address to see the styling options. The first answer after a pause can take about 10 seconds while the model loads.

Step 9. Start over (optional)

To discard everything the CLI has indexed, delete the file eagle-eye/data/eagle-cli.sqlite. The demo keeps its own file, eagle-eye/data/demo.sqlite, which is replaced on every run. Nothing else is affected.

Run the API server (later, when you build the widget)

```
copy .env.example .env
npm run dev:api
```

Edit .env first and set EAGLE_API_KEY to a long random value (16 or more characters). The server listens only on your own computer (127.0.0.1, port 8787) unless you change that.

4. Will these commands break my website?

Worry	Answer
Does it change my website's code or files?	No. Eagle's Eye never opens, edits or deletes your website's project files. It lives in its own folder and only reads pages over the web.
Does it change the live website?	No. The crawler can only send **GET** requests, the same kind a browser sends when someone visits a page. It cannot submit forms, log in, edit or delete anything. This is confirmed in the code (packages/crawler/src/fetcher.ts).
Does npm install affect my other projects?	No, not when run inside eagle-eye. It writes only to eagle-eye/node_modules. Run in a different project's folder by mistake, it would install into that project, which is why section 1 comes first.
Does crawling put load on the website?	Very little. It makes at most about 4 requests per second, two at a time, obeys the site's robots.txt and Crawl-delay, and stops at the page limit. For a small site or a shared host, start with --max-pages 30 and avoid peak hours.
Will it show up in the site's logs or analytics?	Yes, as visits from **EaglesEyeBot**. A firewall or security plugin (for example Cloudflare, Wordfence) may block or rate-limit it. That is expected and not harmful. If it happens, allow the bot for your own site.
Can it crawl my local dev server?	Not by default. For safety it refuses localhost and private addresses. Add --allow-private only for your own local site, and never for sites you do not control.
Where does the data go?	Only into eagle-eye/data/eagle-cli.sqlite on your computer. Nothing is uploaded anywhere. If you configure a language model, the passages used for an answer are sent to that model, so choose a local one if privacy matters.
Is anything committed to GitHub automatically?	No. The data folder, node_modules and .env are excluded from git. Nothing is pushed unless you or I run a push.

Be careful with: crawling a site you do not own or have permission to index (check its terms), running commands from the wrong folder, and sharing your .env file.

5. Troubleshooting

Message or symptom	What to do
'node' is not recognised, or version below 24	Install Node.js 24 or newer from nodejs.org, reopen VS Code, run node -v.
Cannot find package.json / missing script 'eagle'	You are in the wrong folder. Go back to section 1.
crawl: Refusing to crawl internal host / private address	Safety guard. Use a public URL, or add --allow-private for your own local site.
crawl: robots.txt disallows crawling	The site asks bots to stay out. Respect that, or change the site's robots.txt if it is yours.
crawl finishes with 0 pages indexed	The site probably draws its content with JavaScript. Run npx playwright install chromium once, then add **--render**. If it still finds nothing, a firewall may be blocking the bot; send me the URL and output.
Could not start Chromium	Run npx playwright install chromium once and try again.
Answers say 'I couldn't find that on this website'	Either the site really does not say it, or the wording differs. Try search to see what was indexed, and send me the question.
Warning: SQLite is an experimental feature	Harmless. It is a Node notice, not an error.